

# Bases de Datos 2

Teoria

Federico Di Claudio

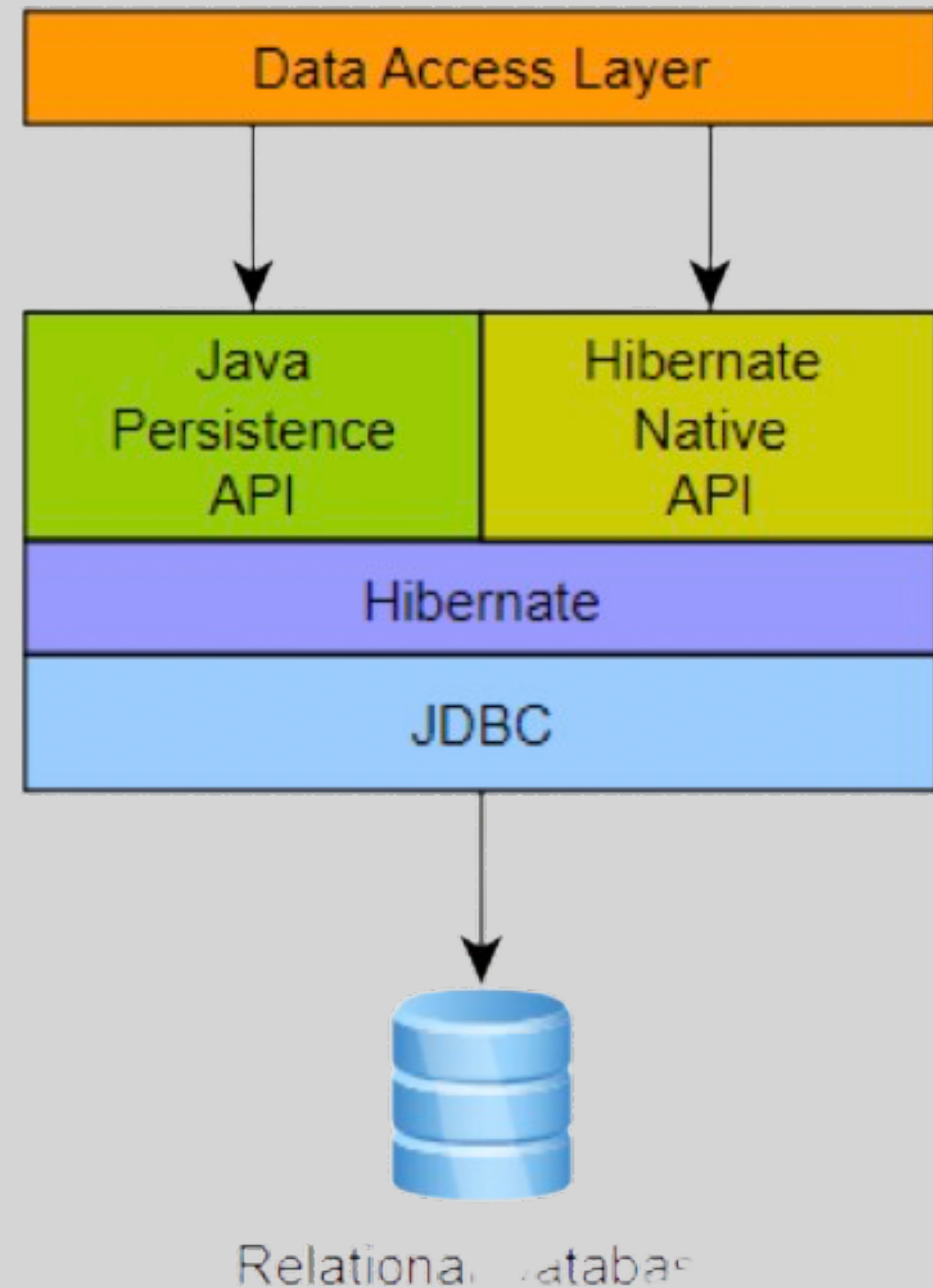
# Organización de la clase

- Repaso
- Spring Data
  - Spring Data JPA
- Repositorios
- Consultas

# Repaso

## Esquema de proyecto actual

- Desarrollado utilizando:
  - JAVA
  - Spring Framework
  - Maven como gestor de dependencias
  - Hibernate y MySQL





# Repaso

## Esquema de proyecto actual

- Implementamos las configuraciones para:
  - Indicar a Hibernate la base de datos y las preferencias de conexión.
  - Utilizamos una clase de configuración.
- Indicar como persistir los objetos (mapeo)
  - Utilizamos anotaciones JPA en las clases

# Nueva solución

## Nueva capa de acceso a datos

- Hoy veremos una solución mas moderna de persistencia.
- Incluiremos una nueva capa de repositorios que estará implementado con Spring Data JPA
- Cambiamos la capa de repositorio, la configuración de conexión y nada más!

# Spring Data





# Spring Data

## Introducción

- **Spring Data** es un framework de Spring que proporciona una **abstracción** sobre diferentes tecnologías de persistencia de datos, incluidos los ORM.
- Simplifica el desarrollo de aplicaciones que interactúan con bases de datos utilizando técnicas comunes
- Tenemos una implementación de Spring Data para el tipo de base de datos que vayamos a utilizar

 Spring Data

 Hibernate

JDBC



# Spring Data JPA

## Implementación para bases de datos relacionales

- Spring Data JPA es la implementación concreta de Spring Data para implementar persistencia en bases de datos relacionales mediante JPA.
- Por defecto utiliza Hibernate como ORM.
- Utiliza todas las características de Hibernate y JPA
- Su principal característica es la de simplificar la creación de repositorios

JPA

 Spring Data JPA

 Hibernate

JDBC



Base de datos relacional



# Spring Data JPA

## Introducción

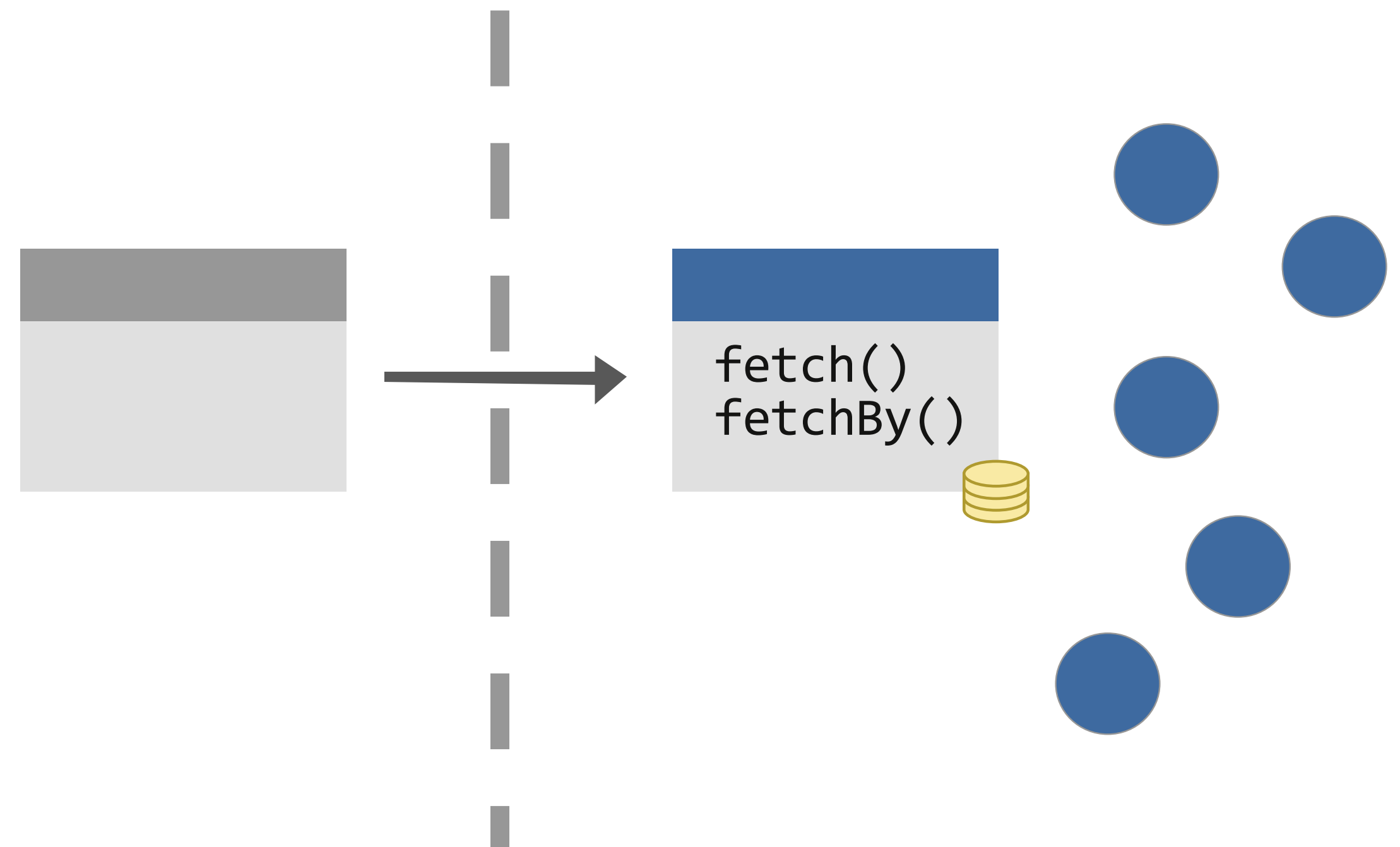
- Spring Data JPA nos proporciona las siguientes características propias:
  - Soporte de configuración de mapeo mediante anotaciones de JPA.
  - Soporte para el manejo de transacciones en la base de datos
  - Soporte para la implementación del patron Repository a traves Spring Data Repositories
  - Soporte para realizar consultas en JPQL, HQL, Criteria y SQL

# Repositories

# Patrón Repository

## Separando la BD del modelo

- El patrón Repository es un patrón de diseño que actúa como una capa intermedia entre la lógica de negocio y la capa de persistencia
- A diferencia del patrón DAO no representa la capa de persistencia, sino la colección de objetos persistentes.
- Se enfoca en el “fetch” - aprovecha los updates de ORM

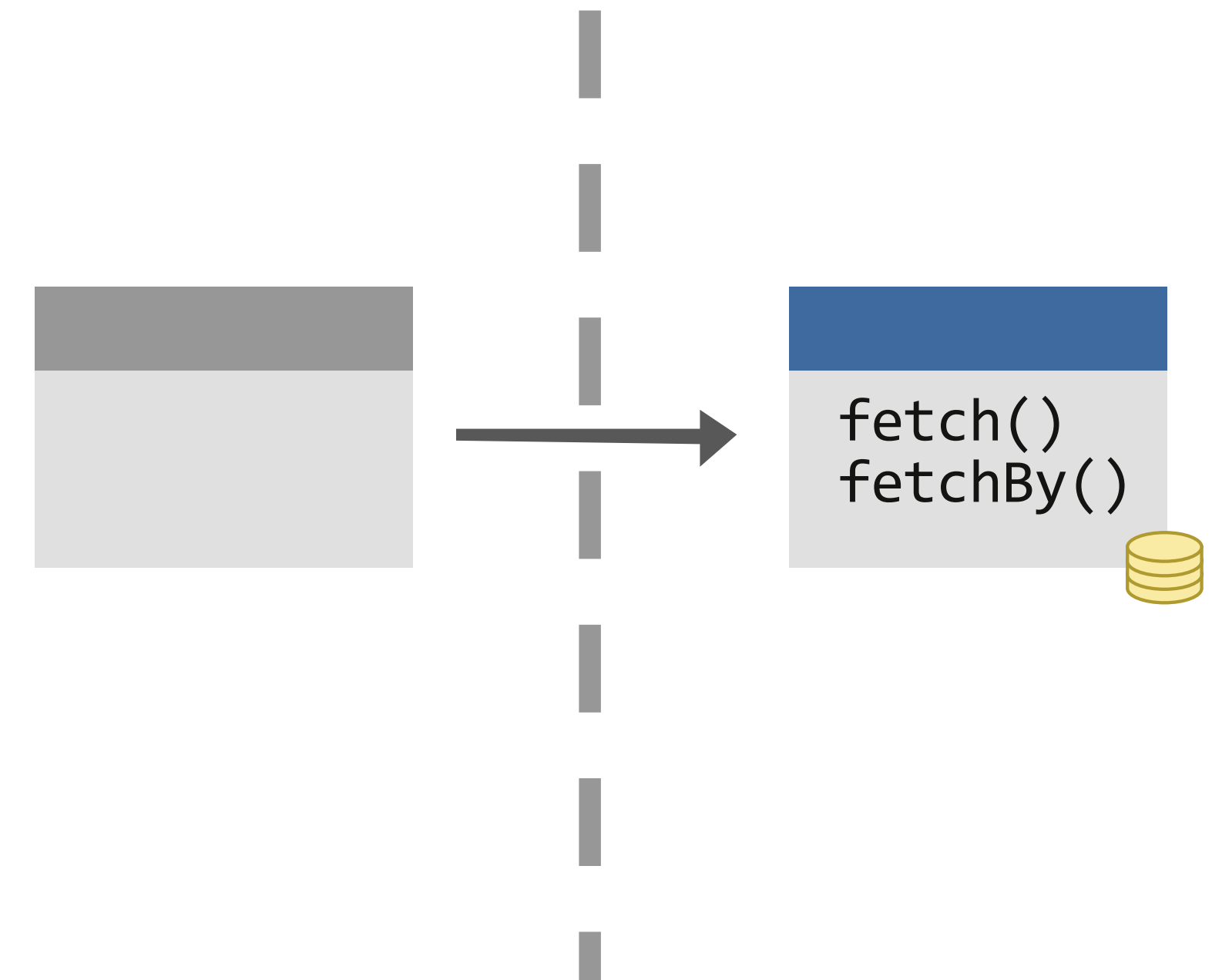




# Spring Data Repositories

## Introducción

- Los **Spring Data Repositories** son un componente de Spring Data, una forma de crear repositorios con métodos de consulta sin la necesidad de escribir su implementación.
- Proporciona una forma automatizada de implementar las **operaciones CRUD**
- Aprovecha la capacidad de Spring de utilizar **reflection**
- Se basa en el **patron repository**



# Spring Data Repository

## Implementación

- En lugar de escribir código repetitivo estos permiten **describir los métodos de acceso a datos** y este, mediante **reflection**, genera su correspondiente implementación
- En tiempo de ejecución el framework detecta estos repostories y genera un **proxy dinámico**
- Este intercepta las llamadas al repositorio y genera las consultas correspondientes

```
select * from Bank where id = :id

Bank findById(Long id);
```

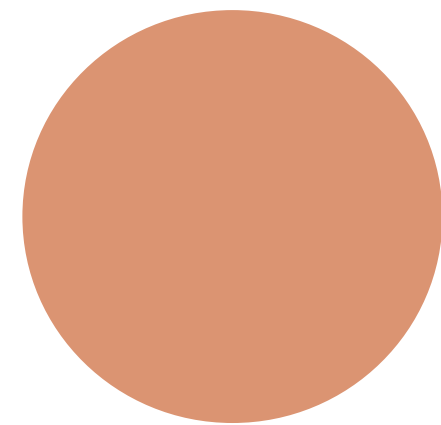
```
select * from Bank order by name

List<Bank> findAllOrderByName();
```

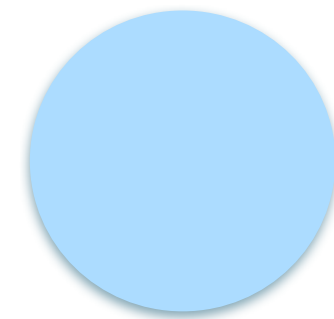
```
insert into Bank (...) values (...)

Bank save(Bank bank);
```

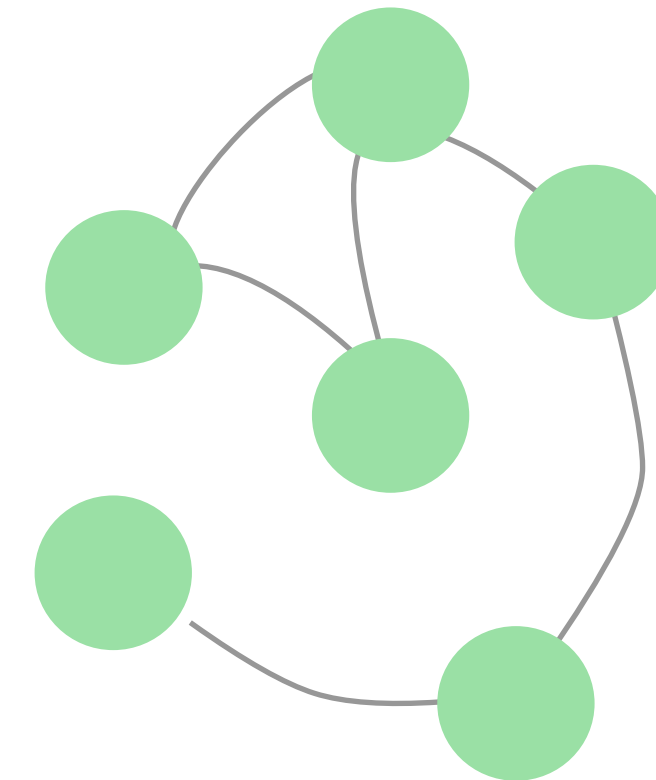
servicio



repositorio



modelo





# Repositorios de Spring Data JPA

# Spring Data Repositories

## Introducción

- Facilita el acceso a datos al proporcionar una nueva capa de abstracción sobre nuestra tecnología de persistencia de datos.
- Nos permite la creación de repositorios con métodos de consulta sin la necesidad de escribir la consulta directamente.
- Se basa en que la mayoría de las consultas son sencillas y siguen una estructura similar



# Spring Data Repositories

## Interfaces

- Solo debemos extender de una interfaz, e indicar la clase del repositorio y el tipo de clave primaria:
  - CrudRepository<type, id>
  - PagingAndSortingRepository<type, id>
  - JpaRepository<type, id>
- No olvidar marcar estas interfaces como @Repository

[Link Documentación](#)

# Spring Data Repositories

## Metodos heradados

- Disponemos de algunos métodos para hacer algunas operaciones básicas sobre los objetos:
  - *save* -> Persistir o actualizar un objeto
  - *findAll* -> Obtiene todas las instancias de esa entidad
  - *findById* -> Obtiene la instancia que posee el id especificado
  - *delete* -> Elimina un elemento de la base de datos

[Link Documentacion](#)

# Query Methods

## Introducción

- Los repositorios de Spring Data permiten describir los métodos de acceso a los datos a través de métodos abstractos en estas interfaces.
  - Son llamados Query Methods
- Spring Data, utilizando reflection, implementa automáticamente esos métodos.
- Debemos seguir una convención establecida

[Link Documentacion](#)



# Anotación @Query

## Alternativa a Query Methods

- Si los Query Methods no son suficientes tenemos alternativas para implementar estas consultas.
  - Por ejemplo, no hay soporte para realizar operaciones como Group by o Having
- La anotación @Query me permite definir la consulta a realizar en el lenguaje JQPL (alternativa similar a HQL)

[Link Documentacion](#)

# Anotación @Query

## Con consultas SQL nativas

- Si aun no puedo realizar la consulta con JPQL, o no lo prefiero, puedo utilizar la anotación @Query para realizar la consulta en SQL
  - Se define la propiedad nativeQuery = true

# Custom Repositories

## Implementar las interfaces nosotros mismos

- Si aun no me es suficiente o quiero un mayor control sobre la consulta puedo realizar implementaciones de los métodos
- Se realizan a traves de una interfaz paralela que luego será padre de mi repositorio.
- Allí puedo tener un mayor control sobre como se realiza la consulta.
- Se necesita para casos muy específicos.